

★ I. scikit-learn vs statsmodels: Propósito y Aplicaciones

En el ecosistema Python, **scikit-learn** y **statsmodels** emergen como herramientas complementarias para modelado de datos, cada una con una filosofía distinta:

scikit-learn es el estándar para *machine learning práctico*: desde regresiones hasta deep learning, su diseño prioriza la creación de modelos predictivos robustos, con herramientas integradas para validación cruzada, tuning de hiperparámetros y pipelines de producción.

statsmodels, en cambio, es la navaja suiza del *análisis estadístico formal*. Ofrece diagnósticos completos (tests de hipótesis, heterocedasticidad, autocorrelación) y resultados detallados tipo R, ideales para investigación o cuando la interpretabilidad es clave.

La elección no es excluyente: muchos flujos de trabajo usan **statsmodels** para exploración inicial y **scikit-learn** para implementación final. La diferencia radica en el **propósito** (explicación vs predicción) y el **nivel de detalle técnico** requerido.

★ II. scikit-learn: Visión General

Enfoque en rendimiento y escalabilidad: scikit-learn es ideal para aplicaciones de *machine learning* a gran escala. Está optimizado para rendimiento (usando Cython para acelerar

operaciones numéricas) y puede manejar grandes conjuntos de datos de manera eficiente.

Interfaz uniforme: Todos los algoritmos de *machine learning* en scikit-learn siguen una estructura similar con métodos como `.fit()`, `.predict()` y `.score()`, lo cual facilita el cambio de modelos.

Pipelines y preprocesamiento: Un *pipeline* es una secuencia automatizada que integra transformaciones de datos y modelos. Scikit-learn incluye módulos para construir estos flujos de trabajo, garantizando consistencia entre entrenamiento y producción.

Cython: Lenguaje híbrido Python/C que acelera cálculos numéricos.
Pipeline: Cadena de pasos desde preprocesamiento hasta modelado.

★ III. Ejemplo: scikit-learn Ajusta el Modelo (sin Estadísticas Detalladas)

Cuando usamos scikit-learn para ajustar un modelo (por ejemplo, una regresión lineal), solamente entrega:

- **Coeficiente(s)** → pendiente(s) de la recta
- **Intercepto** → valor donde corta el eje Y
- **Score (R^2)** → porcentaje de variabilidad explicada

Pero **NO entrega directamente:**

- Errores estándar de los coeficientes
- P-valores (para contrastar hipótesis sobre los coeficientes)
- Intervalos de confianza
- Pruebas de bondad de ajuste (como F-test detallado)

- Diagnósticos de residuos (heterocedasticidad, autocorrelación, etc.)

Ejemplo de aplicación básica scikit-learn == >

```
In [5]: from sklearn.linear_model import LinearRegression
import numpy as np
# Generamos datos de ejemplo
np.random.seed(0)
X = np.random.rand(100, 1) * 10
y = 3 * X.squeeze() + 4 + np.random.randn(100) * 2
# Modelo de regresión lineal
model = LinearRegression()
model.fit(X, y)
# Parámetros de la regresión
print("Coeficiente:", model.coef_[0])
print("Intercepto:", model.intercept_)
print("Score (R^2):", model.score(X, y))
```

Coeficiente: 2.987387004280409
 Intercepto: 4.444302154894448
 Score (R^2): 0.9492021559001605

★ IV. Statsmodels: Visión General

Análisis detallado de resultados: Proporciona métricas completas como errores estándar, valores-p y estadísticas de bondad de ajuste. Su salida tipo R incluye tablas ANOVA y pruebas de diagnóstico (Jarque-Bera para normalidad, Durbin-Watson para autocorrelación). A diferencia de scikit-learn que solo devuelve coeficientes básicos, statsmodels entrega un reporte estadístico completo con `.summary()`.

Estadísticas avanzadas: Especializado en modelos econométricos y técnicas como:

- Modelos robustos (para datos con outliers)
- GLM (Modelos Lineales Generalizados)
- Series temporales (ARIMA, GARCH)

Investigación académica: Ideal para publicaciones científicas, con exportación a LaTeX (`.summary().as_latex()`) y validación de supuestos estadísticos. Mientras scikit-learn se centra en predicción, statsmodels permite análisis inferenciales rigurosos.

Ejemplo: Statsmodels y Reporte Completo del Modelo ==>

```
In [6]: import statsmodels.api as sm
# Agregamos constante a X
X_with_const = sm.add_constant(X)
# Modelo de regresión lineal
model = sm.OLS(y, X_with_const)
results = model.fit()
print(results.summary())
```

```

OLS Regression Results
=====
Dep. Variable:          y      R-squared:                0.949
Model:                  OLS    Adj. R-squared:            0.949
Method:                 Least Squares    F-statistic:          1831.
Date:                   Mon, 28 Apr 2025    Prob (F-statistic):    3.18e-65
Time:                   14:54:23    Log-Likelihood:       -210.83
No. Observations:       100    AIC:                  425.7
Df Residuals:           98    BIC:                  430.9
Df Model:               1
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	4.4443	0.387	11.496	0.000	3.677	5.211
x1	2.9874	0.070	42.793	0.000	2.849	3.126

```

=====
Omnibus:                11.746    Durbin-Watson:          2.083
Prob(Omnibus):           0.003    Jarque-Bera (JB):        4.097
Skew:                   0.138    Prob(JB):                0.129
Kurtosis:                2.047    Cond. No.:               10.9
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

★ V. Diferencias en Implementación y Modelos Soportados

Aunque **scikit-learn** y **statsmodels** implementan la misma regresión lineal ordinaria (OLS), sus diferencias surgen en el **nivel de detalle** y en el **tipo de modelos** que soportan. A continuación, se analizan los principales contrastes:

A. Regresión Logística

Objetivo: Modelar probabilidades para clasificación binaria (ej: riesgo crediticio, diagnóstico médico).

scikit-learn: Optimizado para predicción con soporte a regularización (L1/L2).

statsmodels: Entrega odds-ratios, tests de Wald y significancia de variables.

B. Modelos de Series Temporales

Objetivo: Pronosticar valores futuros considerando patrones temporales (ej: ventas, demanda eléctrica).

scikit-learn: Sin soporte nativo (requiere transformaciones manuales).

statsmodels: ARIMA/SARIMAX para dependencia temporal, GARCH para volatilidad.

C. Modelos Lineales Generalizados (GLM)

Objetivo: Modelar relaciones no-lineales en datos no-normales (ej: conteos con Poisson, seguros con Gamma).

scikit-learn: Solo regresión lineal/logística estándar.

statsmodels: Soporta 10+ distribuciones y funciones de enlace.

D. Análisis de Supervivencia

Objetivo: Estimar tiempos hasta eventos (ej: falla de equipos, supervivencia pacientes).

scikit-learn: Mínimas funcionalidades.

statsmodels: Modelos de Cox proporcionales, curvas Kaplan-Meier.

★ VI. Origen de scikit-learn y statsmodels

scikit-learn: Iniciado por David Cournapeau en 2007 como parte de Google Summer of Code. Surgió para llenar el vacío de herramientas de machine learning accesibles en Python, inspirándose en libretos como SciPy. Su desarrollo comunitario lo ha convertido en el estándar de facto para ML en Python, con contribuciones de gigantes como INRIA y Google (Pedregosa et al., 2011).

statsmodels: Creado por Jonathan Taylor en 2009 (basado en su trabajo previo con R), nació como respuesta a la necesidad de análisis estadísticos rigurosos en Python, especialmente en econometría. Su diseño replicó intencionalmente el enfoque de paquetes como Stata y SAS (Seabold & Perktold, 2010).

Curiosidad: Ambas librerías comparten raíces académicas pero evolucionaron para servir públicos distintos - scikit-learn adoptó el "practical ML" de Kaggle, mientras statsmodels mantuvo el rigor de la estadística matemática tradicional.

Referencias:

- Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830.
- Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference (SciPy 2010)*.

★ VII. Métricas de Evaluación: scikit-learn vs statsmodels

En el análisis de regresión, una **métrica** es una medida cuantitativa que permite evaluar el desempeño de un modelo. Dependiendo del enfoque, pueden centrarse en la capacidad predictiva o en la validación estadística de los resultados. Contextualizar adecuadamente estos indicadores es esencial para interpretar correctamente la calidad del ajuste o el poder de predicción del modelo.

Tanto **scikit-learn** como **statsmodels** proporcionan una variedad de métricas para evaluar el rendimiento de los modelos de regresión, aunque su enfoque y la forma en que se presentan pueden diferir.

scikit-learn:

- Se centra en el machine learning y sus métricas están diseñadas para evaluar la capacidad predictiva de los modelos.
- Proporciona una interfaz uniforme para acceder a estas métricas a través de funciones o métodos.
- Métricas comunes:
 - **R^2 (coeficiente de determinación):** Mide la proporción de la varianza en la variable dependiente que es predecible a partir de las variables independientes.
 - **MAE (Error Absoluto Medio):** Promedio de la magnitud de los errores entre predicciones y valores

reales. Robusto a outliers.

- **MSE (Error Cuadrático Medio):** Promedio del cuadrado de los errores. Penaliza más los errores grandes.
- **RMSE (Raíz del Error Cuadrático Medio):** Raíz cuadrada del MSE. Interpretación más directa ya que está en la misma unidad que la variable dependiente.

statsmodels:

- Se enfoca en el análisis estadístico y econométrico, ofreciendo una gama más amplia de métricas y pruebas estadísticas.
- Además de R^2 , MAE, MSE y RMSE, proporciona:
 - **Pruebas de hipótesis:** Evaluación de la significancia estadística de los coeficientes (prueba t para coeficientes individuales, prueba F para significancia global).
 - **Intervalos de confianza:** Rango plausible para los coeficientes de regresión.
 - **Diagnósticos de residuos:** Verificación de supuestos del modelo (normalidad, homocedasticidad, autocorrelación).

Cuándo usar cada métrica:

- Si el objetivo es **predicción**, métricas como MAE, MSE y RMSE son más relevantes, ya que miden directamente la precisión de las predicciones.
- Si el objetivo es **explicación e inferencia estadística**, las pruebas de hipótesis, intervalos de confianza y diagnósticos

de residuos son cruciales.

- El R^2 es útil en ambos casos como medida general de bondad de ajuste.

Es importante destacar que en muchos casos se utilizan múltiples métricas en conjunto para obtener una evaluación completa del modelo.

★ VIII. Diferencias en la Implementación Matemática

Aunque **scikit-learn** y **statsmodels** implementan el mismo modelo de regresión lineal ordinaria (OLS) basado en mínimos cuadrados (minimización de la suma de errores cuadráticos), sus enfoques matemáticos presentan diferencias fundamentales en implementación y filosofía de uso:

- **Nivel de detalle analítico:**
 - scikit-learn optimiza para *predicción*, usando implementaciones vectorizadas altamente optimizadas (LAPACK/BLAS) con enfoque en escalabilidad
 - statsmodels provee descomposiciones matriciales explícitas (QR, SVD) y cálculos paso a paso para *inferencia estadística*, permitiendo diagnóstico detallado
- **Tratamiento de supuestos:**
 - statsmodels incluye tests formales (Jarque-Bera para normalidad, Breusch-Pagan para heterocedasticidad, Durbin-Watson para autocorrelación)
 - scikit-learn delega la validación de supuestos al usuario, priorizando eficiencia computacional

- **Métodos numéricos:**

- Ambas usan álgebra lineal numérica, pero statsmodels ofrece alternativas robustas (M-estimators, RLM) para violaciones de supuestos
- scikit-learn emplea solvers especializados (liblinear para regresión logística) con tolerancias configurables

Nota técnica: Para datasets mal condicionados, statsmodels muestra advertencias explícitas sobre número de condición matricial (`np.linalg.cond`), mientras scikit-learn aplica regularización implícita en algunos algoritmos.

Referencias:

- Buitinck, L., et al. (2013). *API Design for Machine Learning Software: Lessons from scikit-learn*. arXiv:1309.0238.
- VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media. Cap. 5.
- Greene, W.H. (2018). *Econometric Analysis*. 8th ed. Pearson. (Métodos de estimación robusta)

★ IX. Resumen de Usos Recomendados: Cuándo Elegir Cada Librería

✓ Casos ideales para scikit-learn

- Desarrollo de modelos predictivos rápidos y escalables.
- Integración en pipelines de machine learning (preprocesamiento + modelado).
- Implementación en entornos industriales donde prima la eficiencia sobre la interpretación.
- Aplicaciones de Big Data donde la velocidad es crucial.

- Uso de algoritmos avanzados listos para producción (Random Forests, SVM, redes neuronales).

✓ Casos ideales para statsmodels

- Investigaciones científicas y publicaciones académicas.
- Validación de supuestos estadísticos rigurosos (normalidad, homocedasticidad, independencia).
- Modelos econométricos, series de tiempo y modelos de panel.
- Contextos donde se requiere trazabilidad matemática completa y presentación formal de resultados.
- Enseñanza y aprendizaje detallado de técnicas de regresión y métodos inferenciales.

Consejo profesional: En muchos proyectos avanzados, se utilizan ambas librerías de forma complementaria: statsmodels para el análisis exploratorio y validación de supuestos, y scikit-learn para el modelado predictivo final. Esta combinación ofrece lo mejor de ambos mundos.

★ La elección fundamental es: ¿Predicción o Explicación? ★